

はじめてのAPI

データをWeb経由で取得する

rskskmt

1

Webブラウザで受け取っている情報

普段、Webブラウザで URL（例：<https://xxx.ac.jp/>）を入力してEnterキーを押すと、デザインされたWebページ、つまり、ただのテキストではなく、画像など含めてデザインされたもの（レンダリングされたページ）が見えますよね？

ここで質問

Webブラウザにサーバから送信されるのは、以下のどちら？

- レンダリング済みの画面？
- レンダリングに必要なデータ群？

2

ハイパーリンク

コンピュータもインターネットもまだない1945年に、その後の世界を方向づける論文が発表される

- ヴァネバー・ブッシュ（**Vannevar Bush**）の「**As We May Think**」
 - 戦時中に米国の科学研究全体を束ねた人物が、終戦の年に発表した未来予想
- 問題意識：記録が増えすぎて、**保存はできるが、整理・検索が難しくなった**
- その問題に対する解として構想した機械が **memex**
 - あらゆる資料を人の**連想**のように資料から資料へたどれるマシン（ただし構想どまり）
- 文章同士に紐帯（リンク）をつける構想は、その後ハイパーテキスト（**Hypertext**）として成立
 - このハイパーテキストのリンクがハイパーリンク（**Hyperlink**）

出典: Life 1945年9月10日号・[Wikimedia Commons](#)（パブリックドメイン）



Life誌掲載版の扉絵。額の小型カメラで見たものをそのまま記録する「未来の科学者」——考える仕事を機械に手伝わせる、という特集の顔

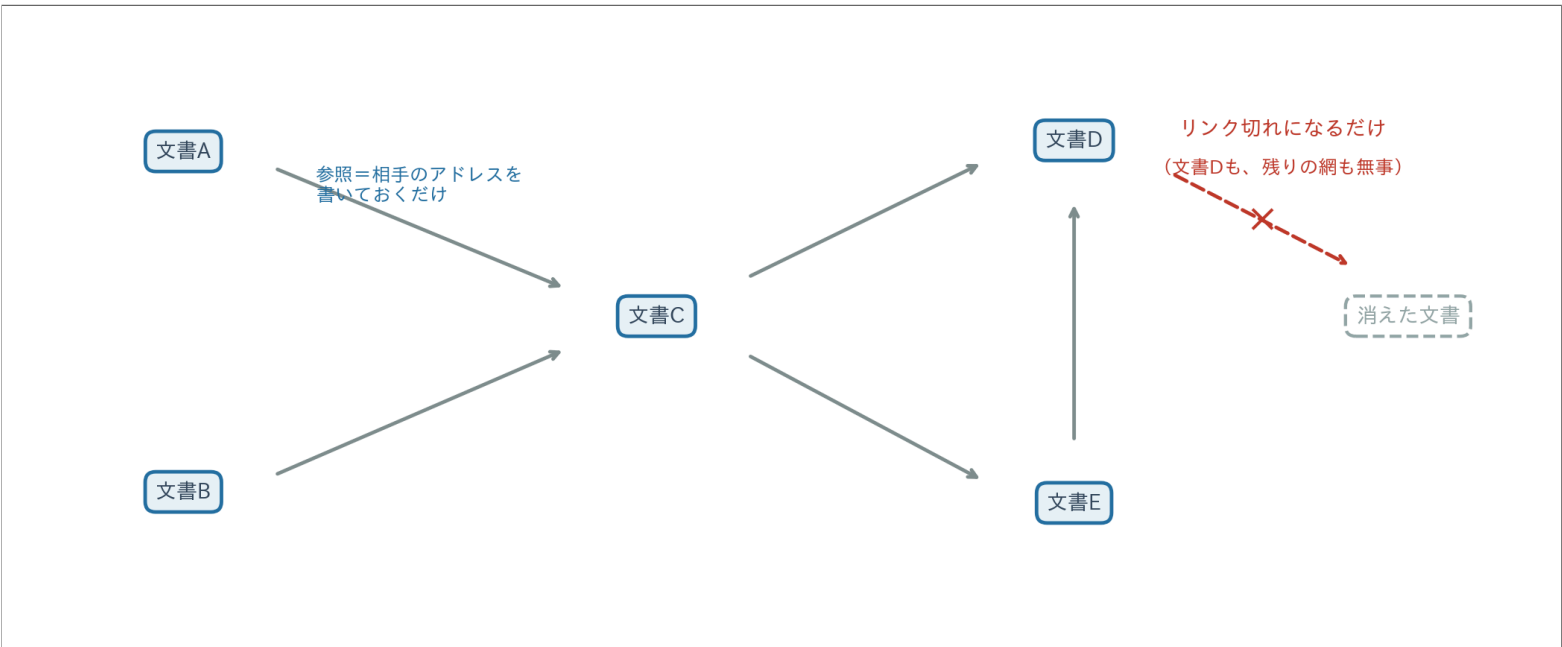
① ハイパーリンクと名付けられたのは1960年代

テッド・ネルソン（**Ted Nelson**）が、memexの構想を発展させて名づけた。

3

ハイパーリンクは疎結合

- Hyperlink**は、文書の中に**他の文書のアドレス（URLなど）**だけを書く
- 相手の文書を取り込まないし、相手の許可も要らない
 - 有向グラフ（**directed graph**）を作るだけ
- このゆるいつなぎ方を**疎結合（loose coupling）**と呼ぶ
- 壊れにくく、誰にも断らずに広げられるため、後日**World Wide Web**として爆発的に普及



4

HTML：Hypertextの1つの実装例

- ハイパーテキストは概念であって実装と仕様は未決定であった
- 具体的に**Hypertext**、**Hyperlink**を実装したのがティム・バーナーズ＝リー（**Tim Berners-Lee**）
 - タグ（**<...>**）でテキストに参照を書き込む**HTML**（**HyperText Markup Language**）を提案
 - 1991年にHypertextのセットであるWWWとともに公開した。

```
1 <h1> HTMLの例 </h1>
2 <p>ハイパーリンクはこんな感じで書く<a href="next.html">次のページ</a>へ</p>
3 
```

href・**src** に書いたURLがハイパーリンク。**<a>** は読者がたどる参照、**** は画像をその場に埋め込む参照

💡 Hypertext ≠ HTML

- 例えば、PDFの中のリンクも、Markdownの **[表示テキスト](URL)** も、**Hypertext**の別実装
- もっと便利な実装を作ることは可能

5

Webブラウザ - HTMLを閲覧するためのアプリ

Webブラウザの仕事（大きく2つ）

1. HTMLを取得する
 - a. ローカルにあるHTMLファイルを読み込む
 - b. ネット（WWW）にあるHTMLを通信して取得（ローカルにキャッシュとして保存）
2. 読み込んだHTMLを解釈して、その命令通りに画面に描画する
 - 画面に描画することを **レンダリング**（**rendering**）という

📌 「Webブラウザ＝インターネット」ではない

- ローカルに保存した **.html** ファイルは、ネットにつながなくても開ける
- メモ帳で前のページの3行を **page.html** と保存し、ダブルクリックすれば確かめられる

6

HTTP - HTMLを通信で取り寄せる取り決め

ローカルのHTMLファイルはダブルクリックで開ける。では、どこかのマシン（サーバ）にあるHTMLはどうやって申請すれば取り寄せられる？この取り寄せのための窓口との申請フォームや窓口の応答マニュアル「こう頼まれたら、こう返す」という手順の取り決めが**プロトコル**（**protocol**）

- HTTP（**HyperText Transfer Protocol**）は、その名のとおり、ハイパーテキストを転送するためのプロトコル
- URLの先頭にある **http://**・**https://** は「HTTPで取り寄せますよ」という宣言
 - **s** は通信を暗号化する版

7

Webサーバ - HTTPでHTMLを戻すサーバ用アプリ

- ネットにはHTML（とその付属ファイル）を沢山保存していて、HTTPで申請されると送り返してくれる専用のコンピュータがある
 - このコンピュータがサーバ
 - ▶ 情報を**serve**するから**server**
 - ▶ サーバは別名ホスト（**host**）と呼ばれる
 - ▶ 要求するWebブラウザはクライアント（**client**）ともいう
 - HTTP用のサーバは、特に **Webサーバ** や **httpd** などと呼ばれる

8

PythonでWebサーバに向かってHTTPを話す

9

【準備】 requests ライブラリ

- requestsはPythonでWebにアクセスするための定番ライブラリ（[公式ドキュメント](#)）。
- 標準ライブラリではないためインストールが必要

インストール

```
Shell
$ pip install requests
```

IMPORT

```
Code
1 import requests
```

requestsを使ってHTTPで要求する

WEBブラウザを使わずにHTTPで生のHTMLを要求

- requestsはHTTP・HTTPS専用のライブラリ
- 引数にURLを渡すだけで、Webサーバの応答が戻り値に来る
- この受信内容（テキスト）をprintで確認

```
Code
1 import requests
2
3 url = "https://www.python.org/" # 見たいページ（何でも可）
4 response = requests.get(url) # WebサーバにそのURLの情報の送信を要求
5
6 print(response.text) # 返信内容（str）
```

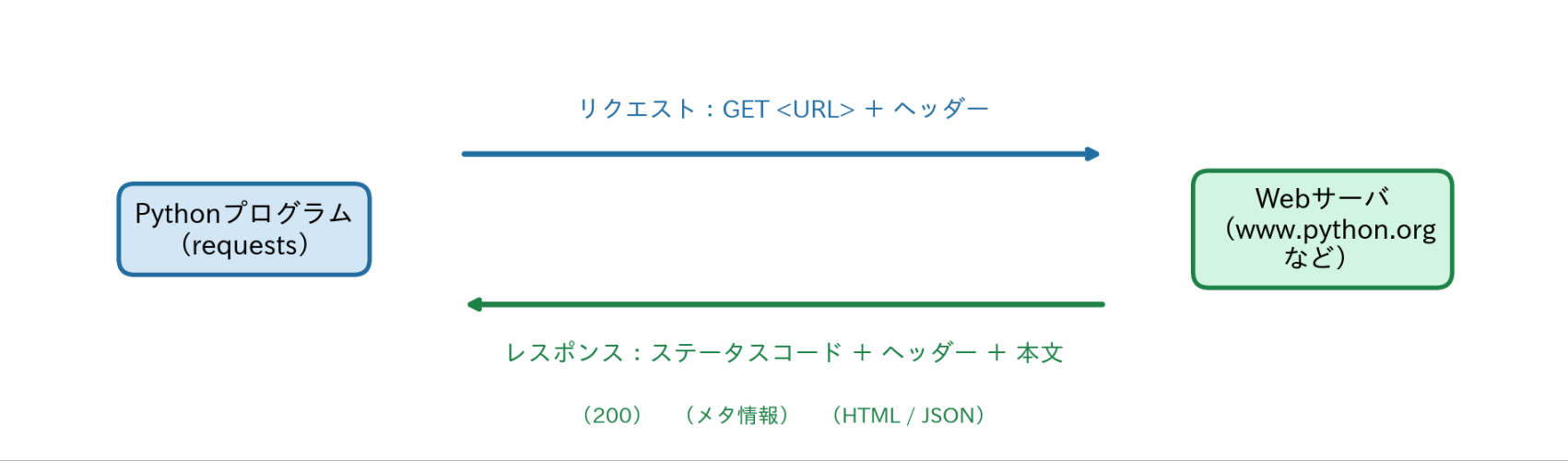
応答内容（長いSTR）

```
1 <!doctype html>
2 <html lang="en">
3 ...
```

コード	役割
<code>requests.get(url)</code>	ブラウザと同じURLへアクセス
<code>response.text</code>	応答の本文（body）を取得

リクエスト（REQUEST）とレスポンス（RESPONSE）

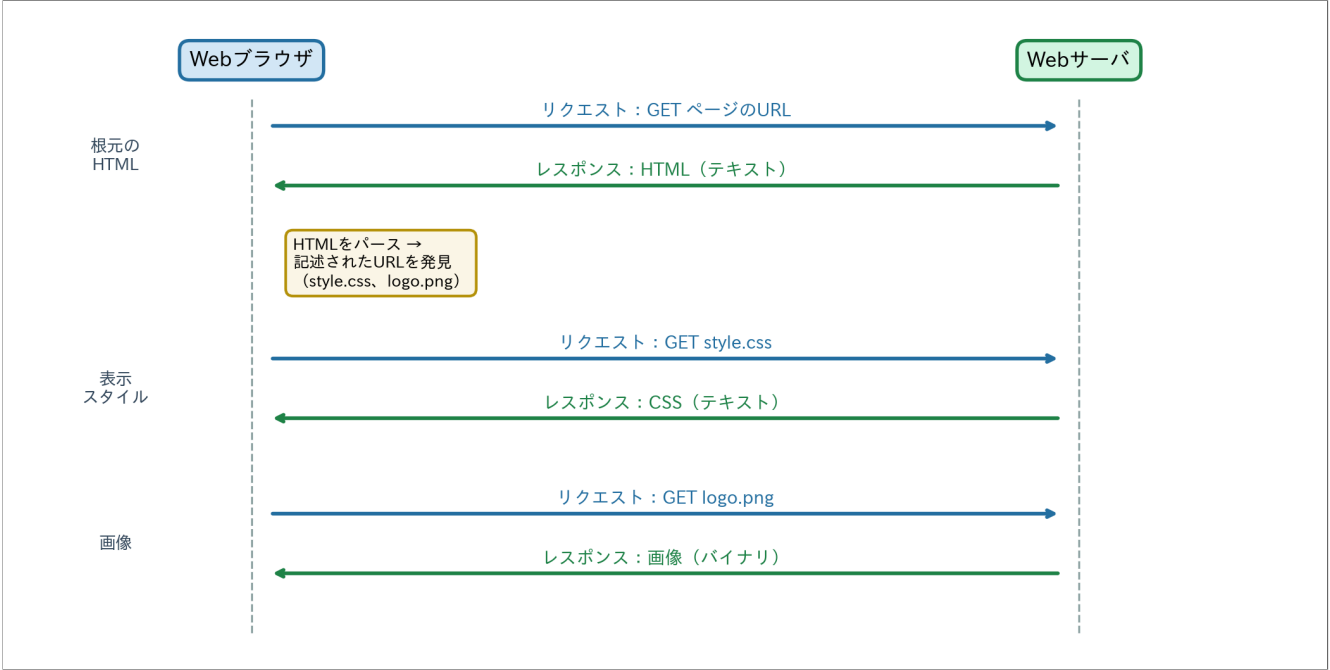
- リクエスト：サーバに返信を要求すること
- レスポンス：リクエストに対するサーバの返信



12

リクエストとレスポンスは1往復

- **requests.get()** は、1つのURLとのrequest → responseを実行する
 - しかし、HTMLの中には、ハイパーリンク（**Hyperlink**）として画像（アイコンも含む）やCSS（表示スタイル用の専用言語）のURLが入っていることが多い
 - Webブラウザは、根元のHTMLを読むと、それをパース（**parse**・分解→解釈すること）する
 - パースされた中に、ハイパーリンクとしてURLが記述されていると、Webブラウザは、さらにその個数分のHTTPリクエストを改めて送る
 - Webサーバはそれらも改めて返答する
- テキストとは限らない。例えば、画像だったらバイナリファイル



13

サーバへ送信した情報の確認

response.request には、サーバの応答だけではなく、送信したリクエストも残っている。

```
Code
1 import requests
2
3 url = "https://www.python.org/"
4 response = requests.get(url)
5 print(response.request.method)
6 print(response.request.url)
7 print(response.request.headers)
```

送信内容：methodは取得を表す **GET**、URLは送信先、request headersは **User-Agent** などの付加情報

14

サーバからの応答を確認

レスポンスは

- **status code** （3桁の数字）
- **response headers** （サーバがつけるメタデータ）
- **body** （本文）

の3点セット。これらを表示して、body以外に何が届いているかを確認する。

Code

```
1 import requests
2
3 url = "https://www.python.org/"
4 response = requests.get(url)
5 print(response.status_code)
6 print(response.headers)
7 print(response.text) # 本文。bodyのこと
```

応答内容（STATUS CODE → HEADERS → BODY の順に表示される）

```
1 200
2 {'Connection': 'keep-alive', 'Content-Length': '11754', ..., 'server': 'nginx', ..., 'content-type': 'text/html; charset=utf-8',
3  <!doctype html>
4  <html class="no-js" lang="en" dir="ltr">
5    ...
```

1行目 が status code（**200**）。**2行目の辞書** が response headers——応答には本文だけでなく、サーバがつけた **名前: 値** の付加情報がこうして必ず一緒に届く。**content-type: text/html** は「bodyはHTML」の意味。**3行目以降** が body

HTTPステータスコード

- 存在しないページを開くと「**404 Not Found**」と表示される
→ この404が **status code**
- **print(response.status_code)** が表示した **200** も、同じstatus code
- サーバは処理の結果を、毎回3桁の番号で応答している

コード	意味
200 OK	正常に取得できた
403 Forbidden	アクセスを拒否した（権限・認証・ヘッダーなど）
404 Not Found	指定したものが見つからない
500 Internal Server Error	サーバ側で処理に失敗した

先頭の数字で5系統に分かれる

系統	意味	例
1xx	処理の途中経過を伝える	100 Continue
2xx	リクエストに成功した	200 OK
3xx	別のURLへ案内する	301 Moved Permanently
4xx	リクエストを受け付けられない	403 Forbidden / 404 Not Found
5xx	サーバ側で処理に失敗した	500 Internal Server Error

読み方： 先頭1桁が大分類、残り2桁が詳しい理由



1. さっきのコードの `url` を下表の値に変更
2. 実行するたびに、以下を `print` して確認
 - `response.status_code`
 - `response.headers["Content-Type"]`

1	Webページ	"https://www.python.org/"
2	タブのアイコン	"https://www.python.org/favicon.ico"

💡 status codeは同じ、Content-Typeだけ違う

- どちらも成功なのでstatus codeは **200**
- 一方、**Content-Type** はWebページが **text/html**、アイコンが **image/x-icon**
- ブラウザは別々のURLへGETし、受け取った部品を組み合わせで画面を作る

HTTPヘッダー

- ヘッダーは、リクエストとレスポンスの両方につく **名前: 値** 形式のメタデータ
- 実際の通信では1行に1つずつ並ぶテキストで、requestsはそれをdictにしている

ヘッダー	実際の値	意味
content-type	text/html; charset=utf-8	bodyの種類と文字コード。ブラウザはこれで描き方を決める
Content-Length	11754	bodyの大きさ（バイト数）
content-encoding	gzip	bodyの圧縮方式（requestsが自動で解凍）
Date	Mon, 20 Jul 2026 16:03:22 GMT	サーバが応答を作った日時（世界標準時）
server	nginx	サーバ側ソフトウェアの名乗り
Connection	keep-alive	この接続を次の往復にも使い回すという宣言
X-Cache	MISS, HIT, HIT	X- で始まる名前は標準外の独自ヘッダー（これはキャッシュに当たったかの記録）

ヘッダーの名前は**大文字小文字**を区別しない。**content-type** と **Content-Type** は同じヘッダー

① リクエスト側にもヘッダーがある

さっき `response.request.headers` に出てきた **User-Agent: python-requests/...** がその例。「誰が頼んでいるか」の名乗りで、このあとAPIを使うときに自分で設定する場面が出てくる。

HTMLではなくJSONをホストするサーバ

APIとは

- インターフェース（**interface**）＝ 何かと何かが接する界面のこと
- その界面をHTTPとするものが **Web API**
 - URLでリクエストすると、ブラウザ用のHTMLではなく **生のデータ** を返す
 - **API**（**A**pplication **P**rogramming **I**nterface）
- アプリはほぼすべてAPIのクライアント
 - 天気アプリも乗換案内も、裏でAPIに問い合わせている

今回利用するAPI

WIKIMEDIA ANALYTICS API

Wikimedia Foundationが公開している、ページビュー統計のAPI

一旦、以下のURLをブラウザで開いてみよう

https://wikimedia.org/api/rest_v1/metrics/pageviews/top/ja.wikipedia.org/all-access/2025/06/27 

2025年6月27日の記事ランキングが、画面ではなく **JSONテキスト** で表示される。

💡 通信の仕組みは同じ

Python.orgもWikipedia APIも、URLへGETしてresponseを受け取る。違うのは、response bodyが **HTMLかJSON**か。

22

Wikimedia APIはヘッダーが必要

- Wikimedia APIでは、**プログラム名・版・連絡先**を含む情報を送る必要がある（**アクセス方針** )
- この情報をUser-Agentとしてrequest headerに指定してrequestsで送信
→ ここでは、仮に以下の情報を送る

```
1 IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)
```

23

requestにヘッダーをつけてAPIを叩く（要求する）

- 以下を動かすと返ってきたテキストはdictみたいなstr
- このstrは、JSON（**JavaScript Object Notation**）という形式

```
Code
1 import requests
2
3 url = "https://wikimedia.org/api/rest_v1/metrics/pageviews/top/ja.wikipedia.org/all-access/2025/06/27"
4 headers = {
5     "User-Agent": "IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)"
6 }
7
8 response = requests.get(url, headers=headers)
9 print(response.status_code)
10 print(response.headers)
11 print(response.text)
```

24

JSON とは

- **JavaScript Object Notation**
- CSVと同じ構造化テキストの一種で、Webではデータ交換の標準形式として使われる。

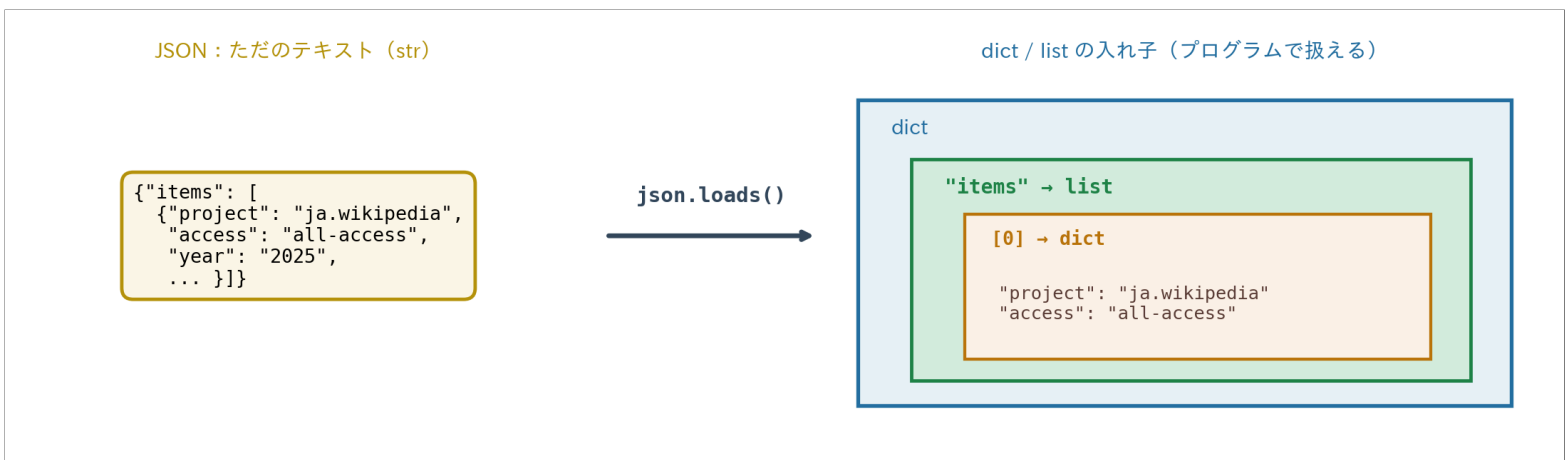
BODYのテキストの中身

```
Code
1 print(response.text)  # 中身は生のJSONテキスト
```

```
1 {"items":[{"project":"ja.wikipedia","access":"all-access",
2      "year":"2025","month":"06", ...}]}
```

JSON形式のSTRをDICTに変換する方法

```
Code
1
2 import json # 標準ライブラリ
3
4 res_dict = json.loads(response.text) # これでdictになる
5 print(type(res_dict), res_dict)
```



25

.json()という便利関数

いちいちjsonライブラリで変換しなくても **`response.json()`** でdictが返る

```
Code
1 import requests
2
3 url = "https://wikimedia.org/api/rest_v1/metrics/pageviews/top/ja.wikipedia.org/all-access/2025/06/27"
4 headers = {
5     "User-Agent": "IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)"
6 }
7
8 response = requests.get(url, headers=headers)
9 data = response.json()
10
11 articles = data["items"][0]["articles"]
12 for article in articles[:5]:
13     print(article["rank"], article["article"], article["views"])
```

26

エンドポイント

APIでは、**エンドポイント**（**endpoint**）という用語がよく出てきます。

用語	意味
API	データ取得などのために公開された仕組み
エンドポイント	APIの中の、具体的な処理を呼ぶ個別のURL

WIKIMEDIAのランキング用エンドポイント

条件をURLに埋め込む。

```
1 https://wikimedia.org/api/rest_v1/metrics/pageviews/top/  
2 {プロジェクト} / {アクセス方法} / {日付}
```

パラメータ	例の値	意味	他の選択肢
プロジェクト	ja.wikipedia.org	日本語Wikipedia	en.wikipedia.org, commons.wikimedia.org など
アクセス方法	all-access	全アクセス	desktop, mobile-app, mobile-web
日付	2025/06/27	指定日	YYYY/MM/DD の任意の日付

① URL構造はAPIごとに異なる

利用時は[公式ドキュメント](#)で、エンドポイントとパラメータを確認

Tweak ランキングの条件を変える



ランキング取得コードを1か所ずつ変えて実行する。

1	2025年の自分の誕生日（月日）のランキング	URL末尾の日付
2	英語版Wikipediaのランキング	ja.wikipedia.org → en.wikipedia.org
3	上位10件を表示	articles[:5] → articles[:10]

💡 予想してから試す

誕生日当日の1位は、事件・行事・人物のどれだと思う？

記事ごとの閲覧数を返すエンドポイント

以下のエンドポイントは、特定の記事について、任意の期間のアクセス数を日単位または月単位で返す（[公式例](#)）。

```
1 https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/  
2 {project} / {access} / {agent} / {article} / {granularity} / {start} / {end}
```

パラメータ	選択肢
project	ja.wikipedia.org など
access	all-access / desktop / mobile-app / mobile-web
agent	all-agents / user / spider / automated
article	任意の記事名（スペースは _ ）
granularity	daily / monthly
start / end	YYYYMMDD 形式

「七夕」のページビューを取得

「七夕」というWikipediaの記事に対する **1年分（2025年）** の日別アクセス数を取得し、統計量を出す例

Code

```
1 import requests
2 import numpy as np
3
4 url = ("https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/"
5       "ja.wikipedia.org/all-access/all-agents/"
6       "七夕/daily/20250101/20251231")
7 headers = {
8     "User-Agent": "IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)"
9 }
10
11 response = requests.get(url, headers=headers)
12 data = response.json()
13
14 # data["items"] は list of dict
15 views = np.array([item["views"] for item in data["items"]])
16
17 print(f"日数:    {len(views)}")
18 print(f"最大:    {views.max()}回")
19 print(f"最小:    {views.min()}回")
20 print(f"平均:    {views.mean():.1f}回")
21 print(f"中央値: {np.median(views):.0f}回")
```

Result

```
日数:    365
最大:    52168回
最小:    88回
平均:    539.1回
中央値:  187回
```

❗ 最大は中央値の約280倍

突出した1日に平均が引っ張られ、中央値との差が大きく開いた。

30

Tweak

どの端末で読まれた？



七夕の取得コード の **all-access** と **all-agents** を変えて実行する（**all-agents** → **user** で、botを除いた**人間の閲覧**だけになる）。

1 PCで読んだ人

desktop / user

2 スマホのブラウザで読んだ人

mobile-web / user

💡 予想してから試す

PCとスマホ、人はどちらで多く読んでいる？ 平均をとって比べると、差は何倍くらい？

❗ **mobile-web**（スマホのブラウザ）を試すと**404**が返ることがある

このAPIは、**該当データが0件（または未収録）の場合も404で返す**（公式のトラブルシューティング）。しかも同じURLでも通ったり404になったりする。404が返ってきたら **response.status_code** を確認する——**応答が成功時と違うことを自分の目で確かめる**のが、エラーへの第一歩。

31

まとめ

- ブラウザの裏側もPythonも、HTTPの **request → response** の1往復
- responseは **status code**・**headers**・**body** の3点セット
- WebページのbodyはHTML、APIのbodyはJSON — 通信の仕組みは同じ
- JSONをdict / listへ変換し、必要な値を取り出す
- APIの個別URLが **エンドポイント**、URL内の条件が **パラメータ**

参照

requests早見表

32

requests早見表

やりたいこと	コード
アクセス	<code>requests.get(url, headers=...)</code>
状態確認	<code>response.status_code</code> (200 = OK)
送ったヘッダー	<code>response.request.headers</code>
返ったヘッダー	<code>response.headers</code>
body (str)	<code>response.text</code>
JSON → Python	<code>response.json()</code>
User-Agent	<code>headers={"User-Agent": "..."}</code>

User-Agentは「自己申告の名札」

User-Agentは、アクセス元ソフトウェアがHTTPリクエストで送る **自己申告の名札**。互換性のための古い名前や固定値が多く、OS・CPU・端末名が実際の環境と一致するとは限らない。

CHROMEの例

見やすいように改行しているが、実際のヘッダー値は1行。

```
1 Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
2 AppleWebKit/537.36 (KHTML, like Gecko)
3 Chrome/<major>.0.0.0 Safari/537.36
```

この文字列の読み方：**ChromeのUser-Agent削減**と歴史的な互換性のため、古いOS・CPU名や複数ブラウザ名が共存する。正確な端末情報ではなく、ソフトウェアが名乗る識別子。